EXPERIMENT NO - 2 : CREATE A BLOCKCHAIN USING PYTHON

AIM : To create and implement a basic Blockchain using Python with Proof-of-Work, transaction handling, block mining, and blockchain validation.

THEORY :

Blockchain is a distributed and immutable digital ledger that stores data in a sequence of interconnected blocks. Each block contains information such as an index, timestamp, proof, transactions, and the hash of the previous block. The connection between blocks using cryptographic hashes helps maintain the integrity and security of the blockchain.

In this experiment, a blockchain is implemented using Python and the SHA-256 cryptographic hashing algorithm.

1. Block Creation

A block is created using the create_block() method. Each block contains:

- Index
- Timestamp
- Proof
- Previous block hash
- Transactions

The first block is called the Genesis Block.

2. Transactions

Transactions contain information about the sender, receiver, and amount. A new block is mined only when pending transactions are available.

3. Proof-of-Work

Proof-of-Work is a computational process used for mining. The program searches for a golden nonce that satisfies the cryptographic puzzle.

In this experiment, the SHA-256 hash must begin with:

000

The nonce is repeatedly changed until this condition is satisfied.

4. Hashing

The **SHA-256** algorithm is used to generate a unique hash for each block. The previous block's hash is stored in the next block, creating a chain of linked blocks.

5. Blockchain Validation

The is_chain_valid() method checks whether:

- The previous_hash of each block matches the actual hash of the previous block.
- The Proof-of-Work condition is satisfied.

If all checks pass, the blockchain is considered valid; otherwise, it is considered invalid.

6. Flask API

Flask is used to provide API endpoints for mining a block, displaying the blockchain, and checking its validity. The experiment can also be demonstrated using Postman.

OUTPUT:

Installation of Flask Library in Google Colab

```
!pip install flask

Requirement already satisfied: flask in /usr/local/lib/python3.13/dist-packages (3.1.3)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.13/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.13/dist-packages (from flask) (8.4.2)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.13/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.13/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from flask) (3.1.8)
```

Importing Required Python Libraries for Blockchain Implementation

```
import datetime
import hashlib
import json
from flask import Flask, jsonify
```

Defining the Blockchain Class and Creating the Genesis Block

```
class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []

        # Create Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0',
            transactions=[]
        )

    # ----------------------------------------------------
    # Create a Block
    # ----------------------------------------------------

    def create_block(self, proof, previous_hash, transactions):

        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': transactions
        }

        self.chain.append(block)

        return block
```

Implementing Transaction Creation and Proof-of-Work Mechanism

```
    # ----------------------------------------------------
    # Get Previous Block
    # ----------------------------------------------------

    def get_previous_block(self):
        return self.chain[-1]

    # ----------------------------------------------------
    # Create Transactions
    # ----------------------------------------------------

    def create_Transactions(self, sender, receiver, amount):

        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        self.transactions.append(transaction)

        return transaction

    # ----------------------------------------------------
    # Proof of Work - Golden Nonce
    # ----------------------------------------------------

    def proof_of_work(self, previous_proof):

        new_proof = 1

        while True:

            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
```

Implementing Golden Nonce Verification and Blockchain Hashing

```
            # Cryptographic puzzle:
            # Hash must start with 000

            if hash_operation[:3] == '000':
                return new_proof

            new_proof += 1

    # ---------------------------------------------------
    # Hash a Block
    # ---------------------------------------------------

    def hash(self, block):

        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()

        return hashlib.sha256(encoded_block).hexdigest()

    # ---------------------------------------------------
    # Check Blockchain Validity
    # ---------------------------------------------------

    def is_chain_valid(self, chain):

        if len(chain) == 0:
            return False

        previous_block = chain[0]

        block_index = 1

        while block_index < len(chain):

            block = chain[block_index]
```

Implementing Blockchain Validation Using Previous Hash and Proof-of-Work

```
                # Check previous hash

                if block['previous_hash'] != self.hash(previous_block):
                    return False

                # Check Proof of Work

                previous_proof = previous_block['proof']
                proof = block['proof']

                hash_operation = hashlib.sha256(
                    str(proof**2 - previous_proof**2).encode()
                ).hexdigest()

                if hash_operation[:3] != '000':
                    return False

                previous_block = block
                block_index += 1

            return True
```

Initializing the Blockchain and Displaying the Genesis Block

```
blockchain = Blockchain()
```

```
print(blockchain.chain)
```

```
[{'index': 1, 'timestamp': '2026-08-27 02:48:10.402269', 'proof': 1, 'previous_hash': '0', 'transactions': []}]
```

Implementing the Block Mining Function with Transaction Verification

```
def mine_block():

    # Check whether transactions exist

    if len(blockchain.transactions) == 0:
        return {
            'message': 'No transactions available. Block cannot be mined.'
        }

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    # Find Golden Nonce

    proof = blockchain.proof_of_work(previous_proof)

    # Hash of previous block

    previous_hash = blockchain.hash(previous_block)

    # Copy current transactions into the block

    transactions = blockchain.transactions.copy()

    # Remove transactions before adding/mining the block

    blockchain.transactions.clear()

    # Create new block

    block = blockchain.create_block(
        proof,
        previous_hash,
        transactions
    )
```

```
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return response
```

Implementing Blockchain Retrieval and Validity Checking Functions

```
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return response
```

```
def is_valid():

    valid = blockchain.is_chain_valid(blockchain.chain)

    if valid:
        return {
            'message': 'All good. The Blockchain is valid.'
        }

    else:
        return {
            'message': 'Houston, we have a problem. The Blockchain is not valid.'
        }
```

```
print(mine_block())
```

```
{'message': 'No transactions available. Block cannot be mined.'}
```

Creating a Transaction and Successfully Mining Block 2

```
transaction = blockchain.create_Transactions(
    sender='Alice',
    receiver='Bob',
    amount=100
)

print(transaction)
```

```
{'sender': 'Alice', 'receiver': 'Bob', 'amount': 100}
```

```
print(mine_block())
```

```
{'message': 'Congratulations, you just mined a block!', 'index': 2, 'timestamp': '2026-08-27 02:50:26.108038', 'proof': 533, 'previous_hash': '93796f16b98456d1d797b73abe44e48cf533151aad8b33b09f778197d775f065', 'transactions'
```

Displaying the Blockchain and Verifying Its Validity

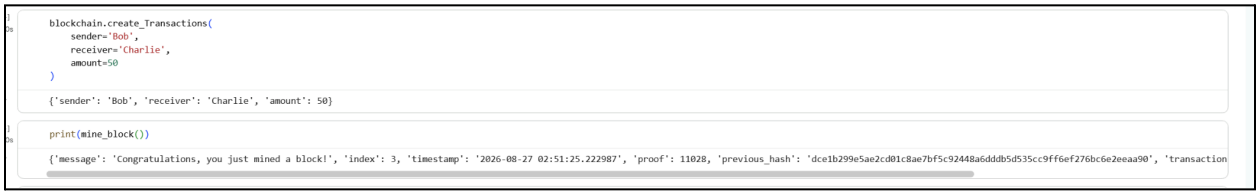
```
print(json.dumps(get_chain(), indent=4))
```


```
{
    "chain": [
        {
            "index": 1,
            "timestamp": "2026-08-27 02:48:10.402269",
            "proof": 1,
            "previous_hash": "0",
            "transactions": []
        },
        {
            "index": 2,
            "timestamp": "2026-08-27 02:50:26.108038",
            "proof": 533,
            "previous_hash": "93796f16b98456d1d797b73abe44e48cf533151aad8b33b09f778197d775f065",
            "transactions": [
                {
                    "sender": "Alice",
                    "receiver": "Bob",
                    "amount": 100
                }
            ]
        }
    ],
    "length": 2
}
```


```
print(is_valid())
```

```
{'message': 'All good. The Blockchain is valid.'}
```

Creating a Second Transaction and Successfully Mining Block 3



Displaying the Updated Blockchain Containing Three Blocks

```
print(json.dumps(get_chain(), indent=4))
```


```
{
    "chain": [
        {
            "index": 1,
            "timestamp": "2026-08-27 02:48:10.402269",
            "proof": 1,
            "previous_hash": "0",
            "transactions": []
        },
        {
            "index": 2,
            "timestamp": "2026-08-27 02:50:26.108038",
            "proof": 533,
            "previous_hash": "93796f16b98456d1d797b73abe44e48cf533151aad8b33b09f778197d775f065",
            "transactions": [
                {
                    "sender": "Alice",
                    "receiver": "Bob",
                    "amount": 100
                }
            ]
        },
        {
            "index": 3,
            "timestamp": "2026-08-27 02:51:25.222987",
            "proof": 11028,
            "previous_hash": "dce1b299e5ae2cd01c8ae7bf5c92448a6dddb5d535cc9ff6ef276bc6e2eeaa90",
            "transactions": [
                {
                    "sender": "Bob",
                    "receiver": "Charlie",
                    "amount": 50
                }
            ]
        }
    ],
    "length": 3
}
```


Mining a New Block Using the Functions Menu

CONCLUSION :

The Blockchain was successfully created and implemented using Python. The experiment demonstrated transaction creation, Proof-of-Work using a golden nonce, block mining, SHA-256 hashing, blockchain display, and chain validation. Blocks were mined only when transactions were available, and the blockchain was successfully verified as valid. Thus, the experiment provided a practical understanding of the basic working and security principles of Blockchain technology.